Conference Paper Title*

*Note: Sub-titles are not captured for https://ieeexplore.ieee.org and should not be used

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

3rd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

5th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

6th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—This document is a model and instructions for LaTeX. This and the IEEEtran.cls file define the components of your paper [title, text, heads, etc.]. *CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract.

Index Terms—component, formatting, style, styling, insert.

I. Introduction

A. Background and Motivation

IoT networks now connect household appliances, industrial controllers, and public-service devices that generate markedly different traffic patterns. A compromised endpoint can provide a route to other devices or services. Identifying malicious traffic is therefore only part of the problem. A useful network intrusion detection system (NIDS) must distinguish attack categories, not merely separate benign from malicious flows. CIC-ToN-IoT provides labeled IoT network flows for this multiclass setting, although its heterogeneous features and sharply unequal class frequencies remain challenging for machine-learning models [1], [2].

B. Challenges

High dimensionality and class imbalance create related modeling problems. Some traffic features are redundant or carry little signal, adding complexity without improving class separation. Meanwhile, majority classes can dominate training; high accuracy may coexist with poor detection of rare attacks. Accuracy alone hides this. Preprocessing introduces a separate risk. If normalization or other data-dependent steps are applied before splitting, information from held-out flows can enter training and make test results look stronger than they are [3], [4].

Identify applicable funding agency here. If none, delete this.

An Autoencoder can learn a compact, nonlinear representation [5], [6], while Light Gradient Boosting Machine (LightGBM) is well suited to tabular intrusion data [7]–[9].

C. Approach and Contributions

Our pipeline begins with all 5,351,760 raw CIC-ToN-IoT flow records. After invalid rows and non-informative fields are removed, 69 traffic features remain. We use a stratified 80:20 train–test split and fit MinMax scaling only on the training partition. The Autoencoder maps the 69 features to a 16-dimensional latent representation, which LightGBM classifies under four training-data treatments: no imbalance handling, class weighting, random upsampling, and random downsampling. Every scenario uses the original, untouched test distribution. We also train LightGBM directly on the 69 normalized features to measure how much discriminative information the latent representation retains.

We make three contributions. First, we implement an end-to-end multiclass NIDS pipeline on the full dataset with explicit controls against preprocessing and test-set leakage. Second, we compare four imbalance-handling scenarios using accuracy, macro precision, macro recall, macro F1-score, per-class metrics, and confusion matrices, which exposes their effects on minority attacks. Third, we contrast the 16-feature latent representation with the original 69-feature representation, quantifying the trade-off between dimensionality reduction and discriminative performance rather than assuming that compression necessarily improves classification.

II. Related Work

A. Detection Paradigms

Intrusion detection can be organized by what is monitored and how attacks are recognized. Host-based systems

inspect activity on individual machines, whereas NIDS analyzes traffic flowing between devices. Signature-based detectors match known patterns and work well when attack definitions already exist; anomaly-based detectors instead flag deviations from learned normal behavior and can expose previously unseen attacks, usually at the cost of more false alarms [10], [11]. For IoT traffic, centralized network monitoring avoids requiring an agent on every constrained device [12]. Research has also shifted from binary benign–attack decisions toward multiclass labels that identify attack families [1], [2]. The added detail is operationally useful, but similar traffic signatures and rare classes make the classification problem harder.

B. Feature Reduction

Feature reduction addresses the redundancy found in many network attributes. Feature selection retains a subset of the original attributes, preserving their meaning and making model behavior easier to trace [13], [14]. Feature extraction instead transforms the inputs into a new representation [6], [13]. Principal Component Analysis provides a linear projection, whereas Autoencoders can capture nonlinear dependencies; sparse and stacked sparse variants further constrain or deepen that representation [5], [15], [16]. Several intrusion-detection pipelines therefore place an encoder before a separate classifier [17], [18]. We follow that arrangement but retain an original-feature baseline, since low reconstruction error does not guarantee that the latent space preserves every class boundary.

C. Gradient Boosting and Imbalance Handling

Boosted tree models remain strong candidates for tabular network data. LightGBM builds an ensemble of decision trees efficiently, and prior NIDS studies have combined it with oversampling, feature selection, or optimization [7]–[9], [19]. These combinations matter because imbalance treatments change what the classifier learns: class weights alter each class's contribution without changing the sample set, upsampling adds minority observations, and downsampling removes majority observations [2], [9]. The balanced weight assigned to class c is

$$w_c = \frac{n}{C \cdot n_c}, \tag{1}$$

where n is the number of training samples, C is the number of classes, and n_c is the number of training samples in class c. Each choice can trade majority-class performance for minority-class recall. Comparisons on an unchanged test distribution and with macro-averaged metrics are therefore more informative than accuracy alone. Our experiments hold preprocessing, representation, classifier settings, and test data fixed so that differences among the four imbalance scenarios can be attributed to the training-data treatment.

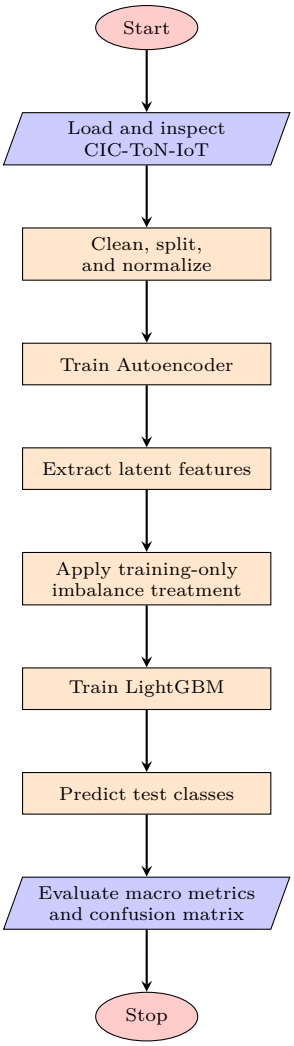


Fig. 1. Leakage-controlled Autoencoder–LightGBM workflow.

III. Methodology

The experimental workflow and model architecture are shown in Figs. 1 and 2. Dataset inspection precedes every learned transformation, and all parameters are estimated without access to the test partition.

A. Dataset

The raw CIC-ToN-IoT CSV contains 5,351,760 network-flow records and 85 columns. The Attack field defines one benign and nine attack classes. Table I reports the pre-cleaning distribution and shows the severe skew: Benign and XSS account for 87.16% of all flows, whereas DDoS and DoS contain only 202 and 145 records, respectively [1], [2].

B. Preprocessing

Identifier and metadata columns, including flow identifiers, endpoint addresses, and timestamps, were excluded together with empirically constant or uninformative fields, leaving 69 numerical features. Removing 1,177 rows with

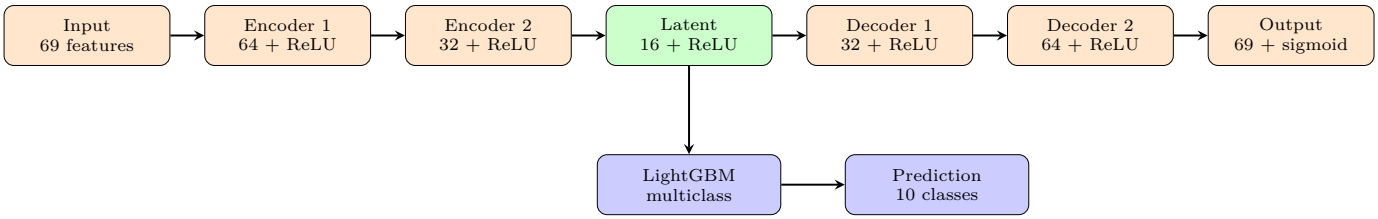


Fig. 2. Autoencoder–LightGBM architecture. The Autoencoder is optimized with MSE and Adam; only the encoder path supplies features to LightGBM.

TABLE I
Raw CIC-ToN-IoT Summary and Class Distribution

Class	Records	Share (%)
Benign	2,515,236	46.998
Backdoor	27,145	0.507
DDoS	202	0.004
DoS	145	0.003
Injection	277,696	5.189
MITM	517	0.010
Password	340,208	6.357
Ransomware	5,098	0.095
Scanning	36,205	0.677
XSS	2,149,308	40.161
Total	5,351,760	100.000

invalid numerical values left 5,350,583 observations. Label encoding was followed by a stratified 80:20 split, producing 4,280,466 training and 1,070,117 test rows. MinMax parameters were fitted only on the training partition and then applied unchanged to both splits. Resampling and class weighting were likewise restricted to training data [3], [4].

C. Autoencoder

The symmetric Autoencoder maps 69 normalized inputs through dense layers of 64 and 32 units to a 16-unit latent layer, then reconstructs them through 32 and 64 units to a 69-unit output. Hidden and latent layers use ReLU; the output uses sigmoid. Training uses mean squared error and Adam for 50 epochs with a batch size of 4096 and a learning rate of 0.001. Mini-batches are reshuffled each epoch, and the checkpoint with the lowest validation loss is retained. Fig. 2 shows both reconstruction and classification paths.

D. Latent Extraction and Imbalance Scenarios

The selected encoder transforms both normalized splits into 16-dimensional arrays, denoted by Z_{train} and Z_{test}. Only Z_{train} and y_{train} are modified by the scenarios in Table II; Z_{test} remains byte-identical throughout. Scenario S2 uses the balanced weights defined in (1).

E. LightGBM and Evaluation

Each scenario trains LightGBM with a multiclass objective, 10 classes, 200 estimators, a learning rate of 0.05, at most 31 leaves per tree, and deterministic seed 42. The fixed estimator count is used without an internal validation split or early stopping. Predictions on the unchanged test set are evaluated using accuracy, macro precision, macro recall, macro F1-score, per-class scores, and a 10×10 confusion matrix [7], [9].

TABLE II
Training-Data Imbalance Scenarios

ID	Training treatment
S1	No imbalance handling
S2	Balanced class weights from (1)
S3	Random upsampling to the largest class
S4	Random downsampling to 116 samples per class

F. Core Algorithm

Algorithm 1 summarizes the shared modeling path after preprocessing. The class-weight expression is referenced rather than repeated.

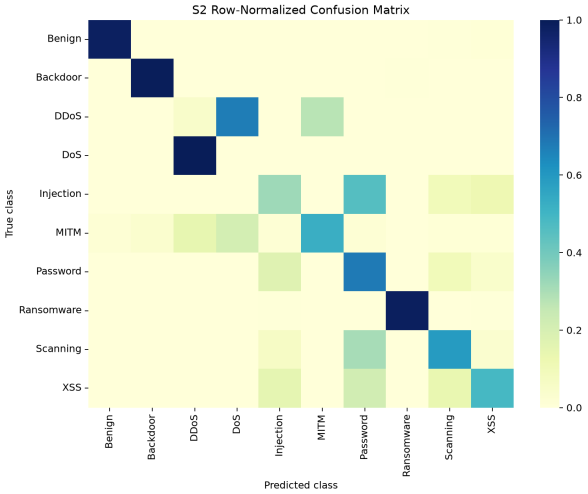
Algorithm 1 Core Autoencoder–LightGBM Pipeline

Require: $X_{\text{train}}^{\text{norm}}$, $X_{\text{test}}^{\text{norm}}$, y_{train}, y_{test}
Require: imbalance scenario $S \in \{S1, S2, S3, S4\}$
Ensure: evaluation result R
Train Autoencoder θ^* on $X_{\text{train}}^{\text{norm}}$ by minimizing MSE
$Z_{\text{train}} \leftarrow \text{Encoder}_{\theta^*}(X_{\text{train}}^{\text{norm}})$
$Z_{\text{test}} \leftarrow \text{Encoder}_{\theta^*}(X_{\text{test}}^{\text{norm}})$
if $S = S1$ then
    Keep Z_{train} and y_{train} unchanged
else if $S = S2$ then
    Assign class weights using (1)
else if $S = S3$ then
    Upsample minority classes in the training set
else if $S = S4$ then
    Downsample majority classes in the training set
end if
$M \leftarrow \text{TrainLightGBM}(Z_{\text{train}}, y_{\text{train}}, S)$
$\hat{y}_{\text{test}} \leftarrow M(Z_{\text{test}})$
$R \leftarrow \text{Evaluate}(y_{\text{test}}, \hat{y}_{\text{test}})$
return R

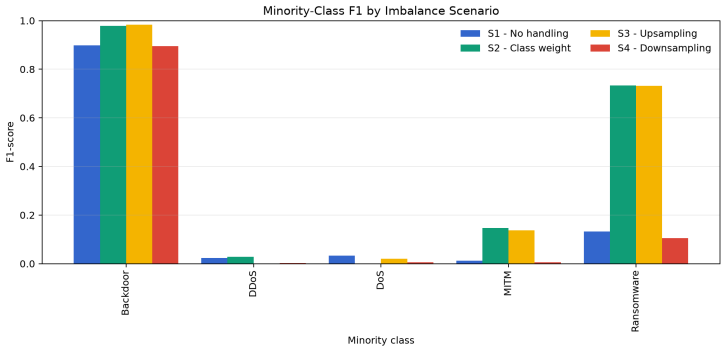
IV. Results and Discussion

A. Scenario-Level Performance

Table III reports full-data results on the unchanged 1,070,117-row test set. S1 achieved the highest accuracy (0.8436) but only 0.3142 macro F1. S2 produced the best macro recall (0.5620) and macro F1 (0.4249). S3 differed from S2 by only 0.0017 macro F1 while requiring 5.07 times as much classifier training time. This similarity follows from random duplication and balanced weighting



(a) S2 row-normalized confusion matrix



(b) Minority-class F1 by scenario

Fig. 3. Class-level results from the full-data experiment. Class weighting improves several rare attacks, but DoS and DDoS remain mutually confused.

TABLE III
Full-Data Performance by Imbalance Scenario

ID	Acc.	Macro P	Macro R	Macro F1	Train (s)
S1	0.8436	0.3204	0.3376	0.3142	104.98
S2	0.7285	0.4106	0.5620	0.4249	104.91
S3	0.7278	0.4103	0.5580	0.4233	532.18
S4	0.6155	0.3050	0.4900	0.2923	1.64

TABLE IV
Latent and Original-Feature Baselines

Features	ID	Acc.	Macro R	Macro F1	Train (s)
Latent-16	S1	0.8436	0.3376	0.3142	104.98
Latent-16	S2	0.7285	0.5620	0.4249	104.91
Original-69	S1	0.8531	0.3401	0.3373	243.98
Original-69	S2	0.7881	0.6483	0.5406	208.44

giving tree training nearly identical relative class contributions. S4 had the lowest macro F1 (0.2923) after retaining only 116 training samples per class.

B. Class-Level Effects

Imbalance handling helped several rare attacks but did not improve every class. From S1 to S2, F1 rose from 0.8991 to 0.9785 for Backdoor, from 0.1328 to 0.7334 for Ransomware, and from 0.0129 to 0.1475 for MITM. However, all 29 DoS test flows were classified as DDoS under S2, while 27 of 40 DDoS flows were classified as DoS; both classes remained below 0.04 F1 in every scenario. The trade-off also affected a majority class: XSS recall fell from 0.9241 under S1 to 0.4898 under S2. S2 is therefore preferred under the macro-metric objective, not for every class or operating requirement. Fig. 3 summarizes these class-level effects.

C. Latent Versus Original Features

Table IV tests whether the 16-dimensional latent representation retains the discriminative information in the 69 normalized features. Under S2, the original features increased macro recall from 0.5620 to 0.6483 and macro F1 from 0.4249 to 0.5406. The latent representation reduced dimensionality by 76.81% and shortened S2 classifier training by 49.7%, but this timing excludes Autoencoder training. H2 is therefore partially supported: the latent features remain useful and compact, but they do not preserve all information needed for classification and should not be claimed to outperform the original features.

D. DoS–DDoS Dataset Limitation

The persistent DoS–DDoS failure is not explained by compression alone because the original-feature baseline did not resolve it. A post-hoc data-quality audit found 145 groups whose 69 features and identifiers were identical but whose labels contradicted each other: one DoS row and one DDoS row per group. These groups contain every DoS record and 71.78% of DDoS records. No deterministic classifier can assign different labels to identical inputs, so this defect imposes a performance limit and is treated as a dataset limitation rather than a methodological contribution.

E. Relation to Published Results

The latent S2 macro F1 of 0.4249 exceeds the 0.392 unaugmented result reported by Ma et al., while the original-69 S2 baseline reaches 0.5406 compared with their 0.506 FSLLM result [2]. These values are indicative rather than head-to-head: their study uses a 70:30 split and 13 selected features, whereas this work uses an 80:20 stratified split and all 69 cleaned features.

References

[1] M. Sarhan, S. Layeghy, and M. Portmann, “Evaluating standard feature sets towards increased generalisability and explainability

of ML-based network intrusion detection," Big Data Research, vol. 30, p. 100359, 2022.
[2] H. Ma, W. Zhang, D. Zhang, and B. Chen, "An IoT intrusion detection framework based on feature selection and large language models fine-tuning," Scientific Reports, vol. 15, 2025.
[3] M. A. Bouke and A. Abdullah, "An empirical study of pattern leakage impact during data preprocessing on machine learning-based intrusion detection models reliability," Expert Systems with Applications, vol. 230, p. 120715, 2023.
[4] L. D. Manocchio, S. Layeghy, M. Gallagher, and M. Portmann, "An empirical evaluation of preprocessing methods for machine learning based network intrusion detection systems," Engineering Applications of Artificial Intelligence, vol. 158, p. 111289, 2025.
[5] T. Zhang, W. Chen, Y. Liu, and L. Wu, "An intrusion detection method based on stacked sparse autoencoder and improved gaussian mixture model," Computers & Security, vol. 128, p. 103144, 2023.
[6] J. Li, H. Chen, M. S. Othman, and L. M. Yusuf, "Enhancing IoT security: A comparative study of feature reduction techniques for intrusion detection system," Intelligent Systems with Applications, vol. 23, p. 200407, 2024.
[7] J. Liu, Y. Gao, and F. Hu, "A fast network intrusion detection system using adaptive synthetic oversampling and LightGBM," Computers & Security, vol. 106, p. 102289, 2021.
[8] A. Munawar, M. Nadeem Ali, A. Qasim, and B.-S. Kim, "GWO-LightGBM: A hybrid grey wolf optimized light gradient boosting model for cyber-physical system security," Computer Modeling in Engineering & Sciences, vol. 145, no. 1, 2025.
[9] A. Wakili and S. Bakkali, "A resilient IoT intrusion detection system using hybrid feature selection and explainable ensemble learning," Results in Engineering, vol. 28, p. 107392, 2025.
[10] M. M. Rahman, S. Al Shakil, and M. R. Mustakim, "A survey on intrusion detection system in IoT networks," Cyber Security and Applications, vol. 3, p. 100082, 2025.
[11] A. Alfahaid, E. Alalwany, A. M. Almars, F. Alharbi, E. Atlam, and I. Mahgoub, "Machine learning-based security solutions for IoT networks: A comprehensive survey," Sensors, vol. 25, no. 11, p. 3341, 2025.
[12] S. R. Hassan, M. U. Tanveer, S. Prajapat, and M. Shabaz, "A comprehensive survey on intrusion detection in internet of medical things: Datasets, federated learning, blockchain, and future research directions," ICT Express, vol. 11, pp. 1291–1310, 2025.
[13] J. Li, M. S. Othman, H. Chen, and L. M. Yusuf, "Optimizing IoT intrusion detection system: feature selection versus feature extraction in machine learning," Journal of Big Data, vol. 11, p. 36, 2024.
[14] O. Achbarou, T. Datsi, O. Bourkoukou, and A. M. El kiram, "Enhanced intrusion detection system using feature selection and hybrid learning models for high performance and efficiency in an IoT environment," Journal of Engineering Research, vol. 14, pp. 953–965, 2026.
[15] W. Yao, L. Hu, Y. Hou, and X. Li, "A lightweight intelligent network intrusion detection system using one-class autoencoder and ensemble learning for IoT," Sensors, vol. 23, no. 8, p. 4141, 2023.
[16] Y. Xiao, Y. Feng, and K. Sakurai, "An efficient detection mechanism of network intrusions in IoT environments using autoencoder and data partitioning," Computers, vol. 13, no. 10, p. 269, 2024.
[17] M. V. Hari Vinayak and T. Jarin, "A hybrid model for detecting intrusions using stacked autoencoders and extreme gradient boosting," Computers & Security, vol. 150, p. 104212, 2025.
[18] A. G. Ayad, M. M. El-Gayar, N. A. Hikal, and N. A. Sakr, "Efficient real-time anomaly detection in IoT networks using one-class autoencoder and deep neural network," Electronics, vol. 14, no. 1, p. 104, 2025.
[19] B. M. Kouassi, A. B. Ballo, K. J. Ayikpa, D. Mamadou, and M. Z. J. Coulibaly, "Top-k feature selection for IoT intrusion detection: Contributions of XGBoost, LightGBM, and random forest," Future Internet, vol. 17, no. 11, p. 529, 2025.